Project2

```
# We start by code to simulate flipping a fair coin 100 times. We use the fact that we can regar
d a bernoulli dis-
# tribution as a binomial distribution with where the number of repetitions n = 1. The number 1
indicates heads.
set.seed(1)
coin_flips <- rbinom(100, 1, .5)
coin_flips
```

```
##   [1] 0 0 1 1 0 1 1 1 1 0 0 0 1 0 1 0 1 1 0 1 1 0 1 0 0 0 0 0 1 0 0 1 0 0 1 1 1
##  [38] 0 1 0 1 1 1 1 1 1 0 0 1 1 0 1 0 0 0 0 0 1 1 0 1 0 0 0 1 0 0 1 0 1 0 1 0 0
##  [75] 0 1 1 0 1 1 0 1 0 0 1 0 1 0 0 0 0 0 1 1 1 1 0 0 1 1
```

```
Data <- data.frame(toss_number = (1:length(coin_flips)) , toss_result = coin_flips)
head(Data, 5)
```

	toss_number <int>	toss_result <int>
1	1	0
2	2	0
3	3	1
4	4	1
5	5	0

5 rows

```
frequencies <- function(vec){
 # get the length of the vector we are using in the calclations.
 len_vec <- length(vec)

 # Create vector to store evolving averages
 avg <- 1:len_vec

 # loop through to compute the averages. We compute the averages by counting the number of
 for(i in  1:len_vec){
     avg[i] <- length(which(vec[1:i]==1))/i
 }
  return(avg)
}

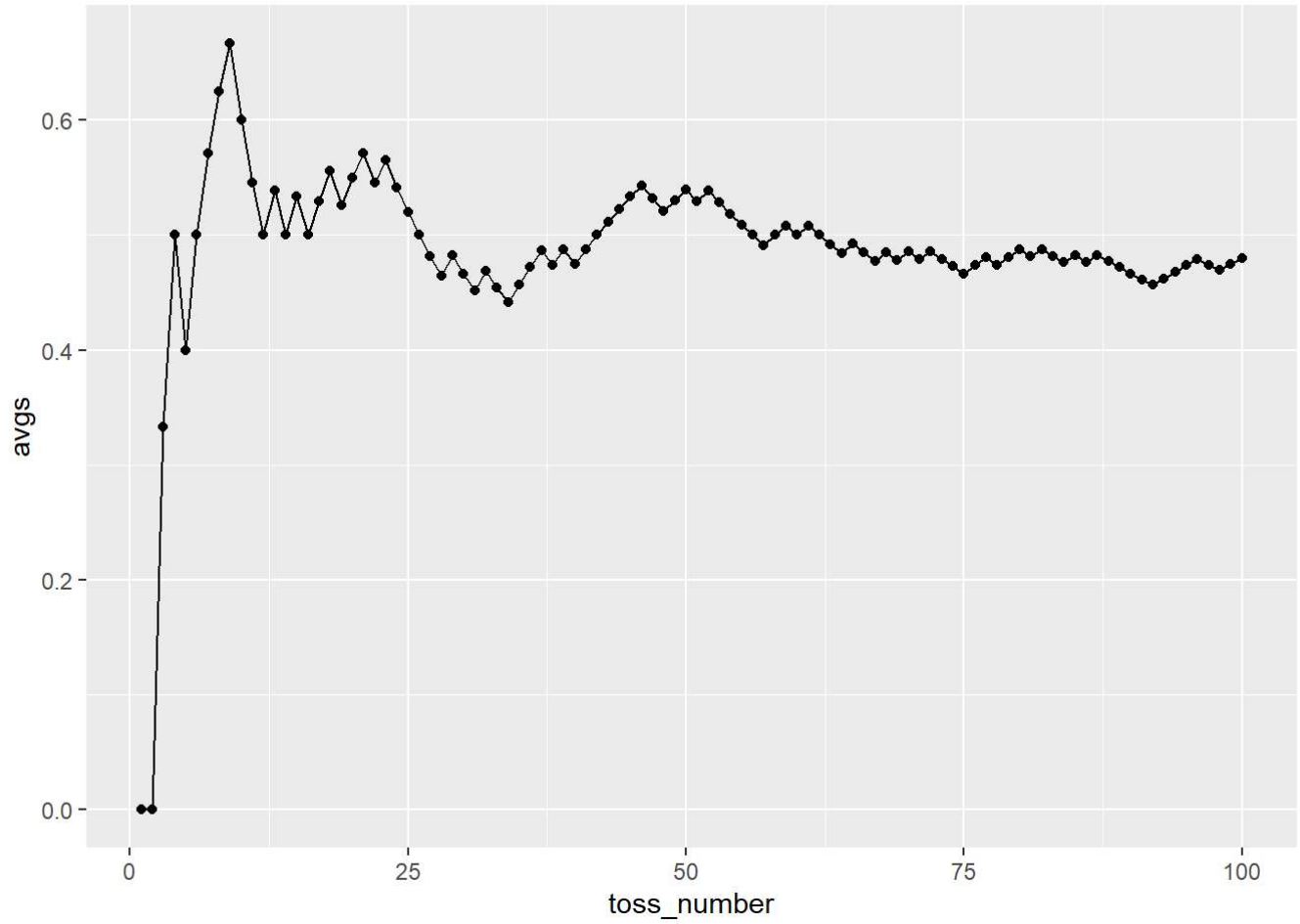
 # We add a new column to the dataframe that contains the aveages
 freq <- frequencies(coin_flips)
 Data$avgs <- freq
 head(Data, 5)
```

	toss_number <int>	toss_result <int>	avgs <dbl>
1	1	0	0.0000000
2	2	0	0.0000000
3	3	1	0.3333333
4	4	1	0.5000000
5	5	0	0.4000000

5 rows

```
library(ggplot2)

ggplot(data=Data, aes(x=toss_number, y=avgs, group=1)) +
  geom_line()+
  geom_point()
```




```
#Set seed for reproducibility
  set.seed(1)

#simulate sample
  sum <- runif(10000, min = 0, max = 1)

#compute the sample average
mean(sum)
```

```
## [1] 0.500168
```

```
estimate_pi <- function(seed = 1, iterations = 100000000) {
  #setting seed for reproducibility
  set.seed(seed)

  #generating the points

  x <- runif(n = iterations, min = 0, max = 1)

  #Computing an approximation for pi /4

  val_of_g = sqrt(1-x^2)

  pi_over_four = mean(val_of_g)

  #estimating pi using formula pi = 4*the mean

  pi <- 4 * pi_over_four
  return(pi)

}
```

```
estimate_pi()
```

```
## [1] 3.141566
```

```
# load required libraries
library('tidyverse')
```

```
## ── Attaching core tidyverse packages ──────────────────────── tidyverse 2.0.0 ──
## ✓ dplyr     1.1.4     ✓ readr     2.1.5
## ✓ forcats   1.0.0     ✓ stringr   1.5.1
## ✓ lubridate 1.9.4     ✓ tibble    3.2.1
## ✓ purrr     1.0.4     ✓ tidyr     1.3.1
## ── Conflicts ────────────────────────────────────────── tidyverse_conflicts() ──
## ✖ dplyr::filter() masks stats::filter()
## ✖ dplyr::lag()    masks stats::lag()
## ℹ Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to becom
e errors
```

```
# Function that mimics the lottery and graphs
lottery <- function(seed = 1, iterations = 15000) {

#Set seed for reproducibility
  set.seed(seed)

#generate n sets of ticket numbers where a ticket number is a set of three
#integers between 1 and 4 inclusive
  num1 <- floor(runif(n = iterations, min = 1, max = 4))
  num2 <- floor(runif(n = iterations, min = 1, max = 4))
  num3 <- floor(runif(n = iterations, min = 1, max = 4))

#Calculate sum of each ticket
  sum <- num1 + num2 + num3

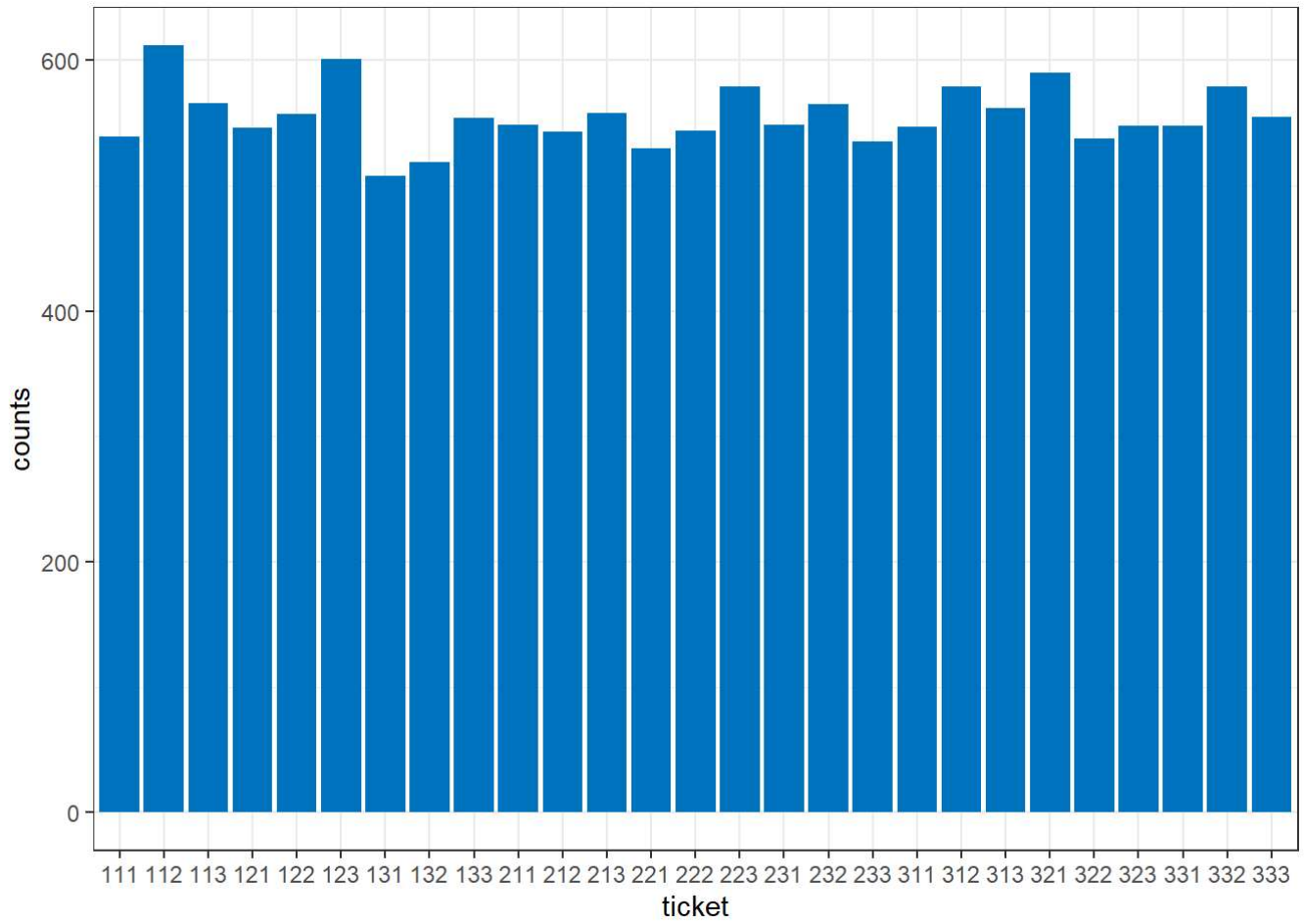
#Create dataframe with ticket numbers in order to make easier to graph
  ticket <- 100*num1 + 10*num2 + num3
  ticket <- as.data.frame(ticket)

#Count occurrences of each ticket number
  df <- ticket %>%
    group_by(ticket) %>%
    summarise(counts = n())


#Graphs occurrences of each ticket number
    df$ticket <- as.factor(df$ticket)
    p <- ggplot(df, aes(x = ticket, y = counts)) +
      geom_bar(fill = "#0073C2FF", stat='identity') +
      theme_bw()
    print(p)

  return(list(df, table(sum)))
}
```

```
lottery()
```




```
## [[1]]
## # A tibble: 27 × 2
##    ticket counts
##    <fct>   <int>
##  1 111       539
##  2 112       612
##  3 113       566
##  4 121       546
##  5 122       557
##  6 123       601
##  7 131       508
##  8 132       519
##  9 133       554
## 10 211       549
## # ℹ 17 more rows
##
## [[2]]
## sum
##    3    4    5    6    7    8    9
##  539 1707 3251 3940 3346 1662  555
```

```r
# Set seed
set.seed(67)

p_x7 <- dbinom(7, size = 15, prob = 0.5)
p_x7
```

```
## [1] 0.1963806
```

```r
# Number of repetitions
n_rep <- 100000

X <- rbinom(n_rep, size = 15, prob = 0.5)

is_seven <- as.numeric(X == 7)

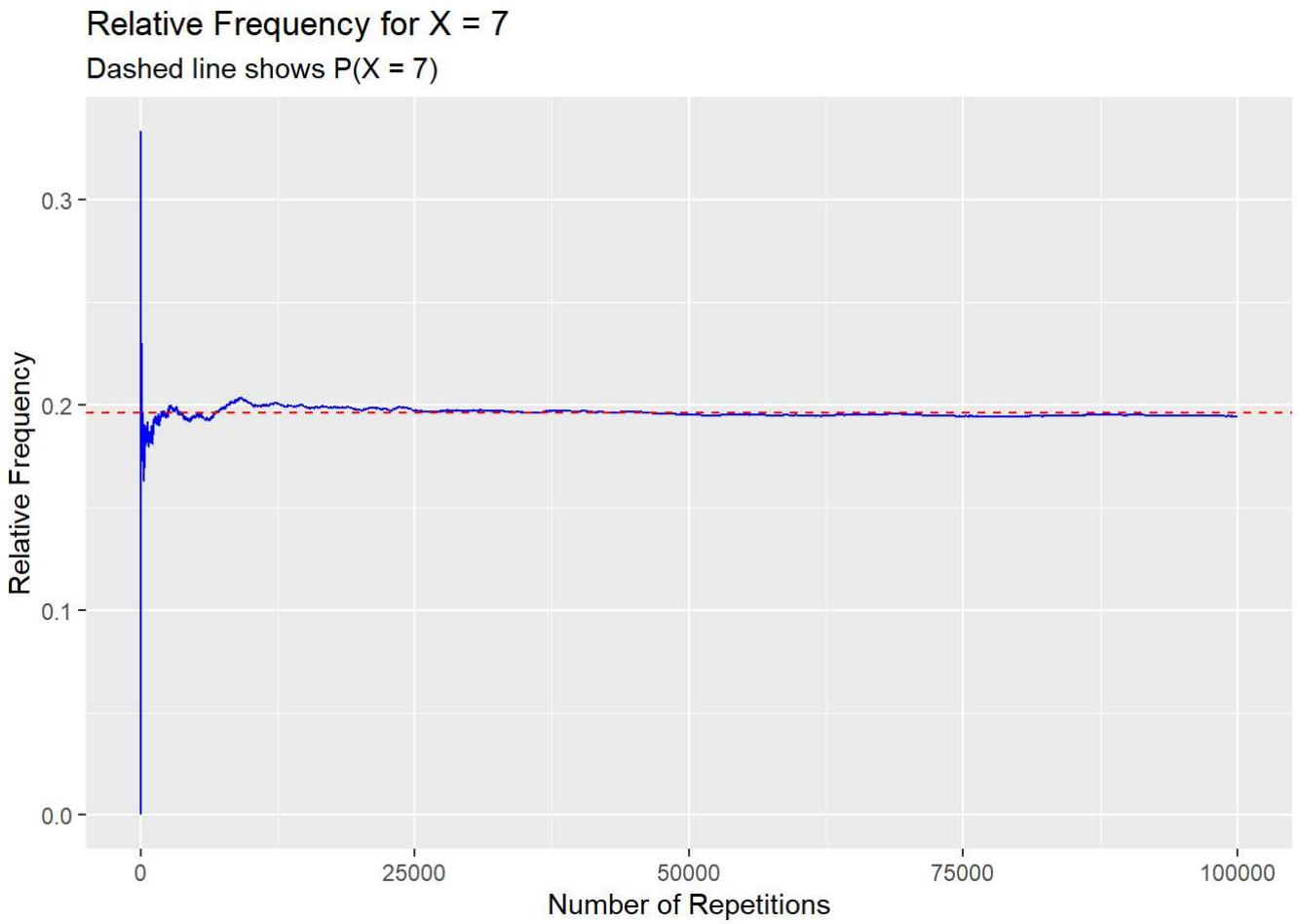
# Function
cumulative_freq <- function(vec) {
  n <- length(vec)
  cum_freq <- numeric(n)
  for(i in 1:n) {
    cum_freq[i] <- sum(vec[1:i]) / i
  }
  cum_freq
}

# Compute the relative frequency
rel_freq_7 <- cumulative_freq(is_seven)

# Create a data frame
coin_data <- data.frame(
  Experiment = 1:n_rep,
  RelativeFrequency = rel_freq_7
)
```

```r
library(ggplot2)

ggplot(coin_data, aes(x = Experiment, y = RelativeFrequency)) +
  geom_line(color = "blue") +
  geom_hline(yintercept = p_x7, linetype = "dashed", color = "red") +
  labs(title = "Relative Frequency for X = 7",
       subtitle = "Dashed line shows P(X = 7)",
       x = "Number of Repetitions",
       y = "Relative Frequency")
```



1.c)

As the number of repetitions increases, the relative frequency converges to the theoretical value P(X=7). This illustrates the frequentist interpretation of probability – probability is the limit of the relative frequency as the number of trials goes to infinity.

```
# Theoretical expected value
E_X <- 15 * 0.5

# Observed average from simulation
observed_avg <- mean(X)

E_X
```

```
## [1] 7.5
```

```
observed_avg
```

```
## [1] 7.50329
```

1.e) The observed average should be very close to 7.5 when the number of repetitions is large. Which supports the law of large numbers.

```
# Set seed
set.seed(67)

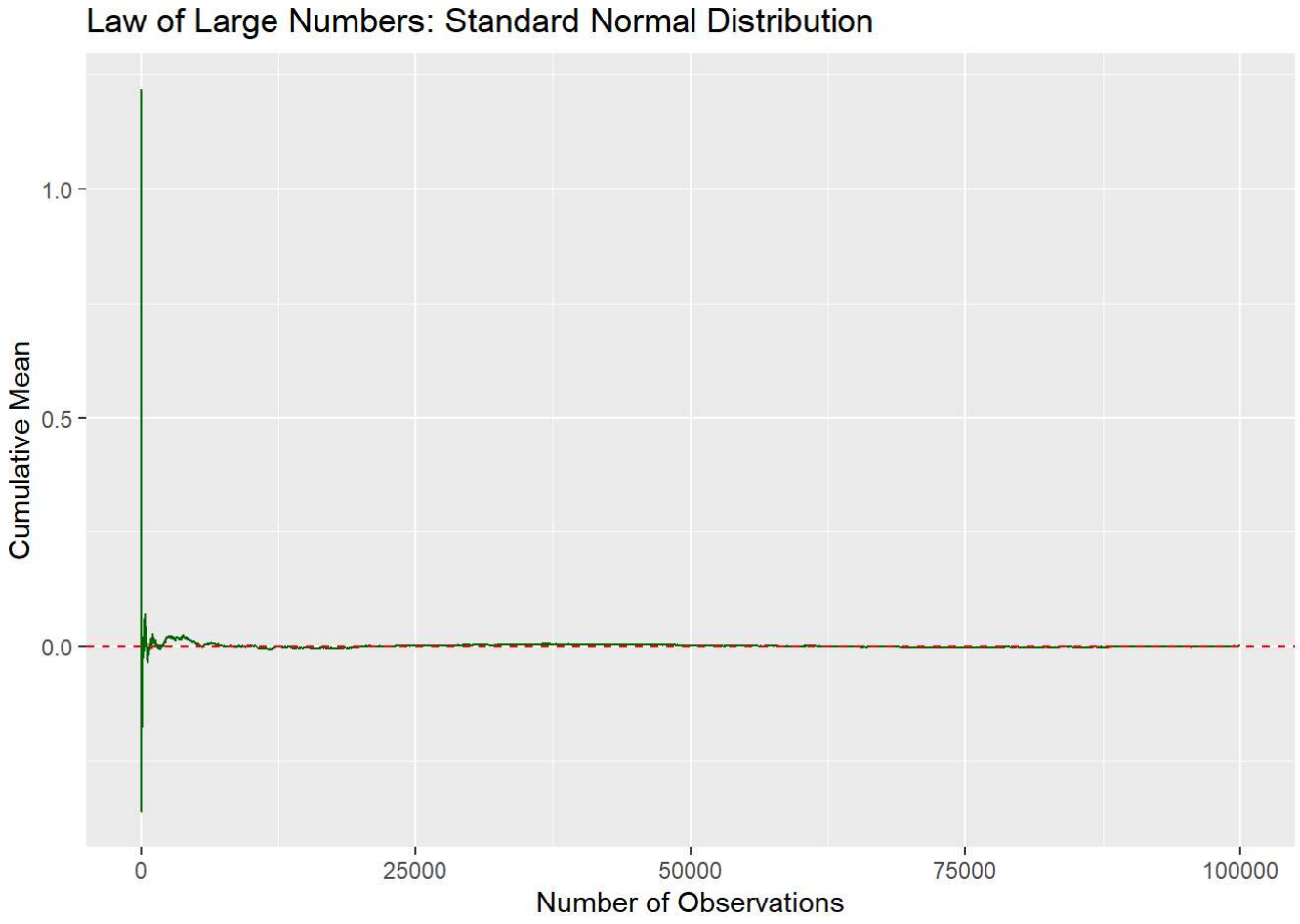
n <- 100000
normal_sample <- rnorm(n)

# Compute cumulative average for norm dist
cum_avg_normal <- cumulative_freq(normal_sample)

cum_avg_normal <- cumsum(normal_sample) / (1:n)

# Create data frame
normal_df <- data.frame(
  Experiment = 1:n,
  CumulativeMean = cum_avg_normal
)

ggplot(normal_df, aes(x = Experiment, y = CumulativeMean)) +
  geom_line(color = "darkgreen") +
  geom_hline(yintercept = 0, linetype = "dashed", color = "red") +
  labs(title = "Law of Large Numbers: Standard Normal Distribution",
       x = "Number of Observations",
       y = "Cumulative Mean")
```



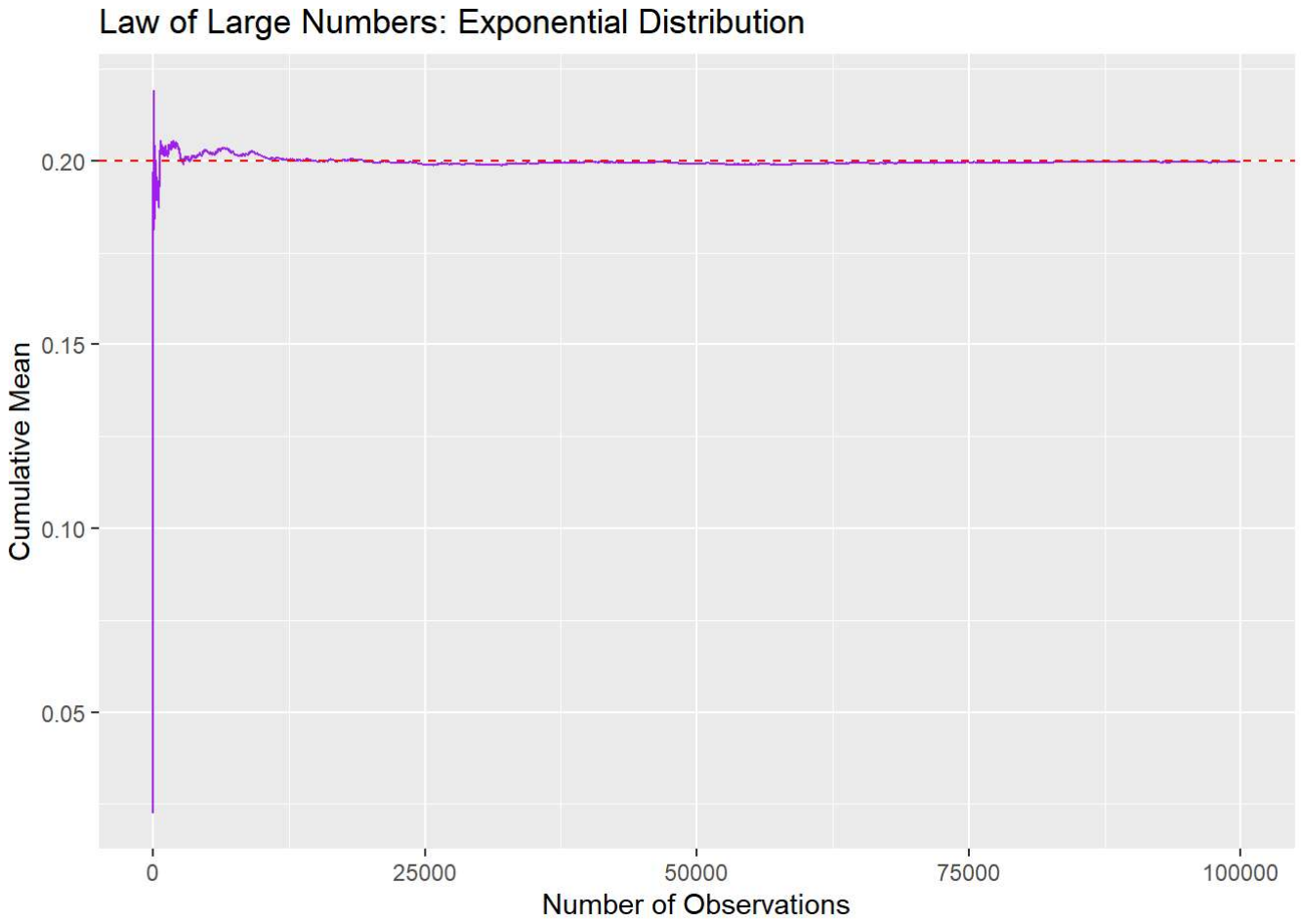
```
# Set seed
set.seed(67)

exp_sample <- rexp(n, rate = 5)

cum_avg_exp <- cumsum(exp_sample) / (1:n)

exp_df <- data.frame(
  Experiment = 1:n,
  CumulativeMean = cum_avg_exp
)

ggplot(exp_df, aes(x = Experiment, y = CumulativeMean)) +
  geom_line(color = "purple") +
  geom_hline(yintercept = 0.2, linetype = "dashed", color = "red") +
  labs(title = "Law of Large Numbers: Exponential Distribution",
       x = "Number of Observations",
       y = "Cumulative Mean")
```



In both cases the cumulative mean converges to the theoretical mean as the sample size increases. Like what the law of large numbers expects.

```
set.seed(67)

lottery <- function(seed = 67, iterations = 15000) {

  set.seed(seed)

  # Generate n sets of ticket numbers
  num1 <- floor(runif(n = iterations, min = 1, max = 4))
  num2 <- floor(runif(n = iterations, min = 1, max = 4))
  num3 <- floor(runif(n = iterations, min = 1, max = 4))

  sum <- num1 + num2 + num3

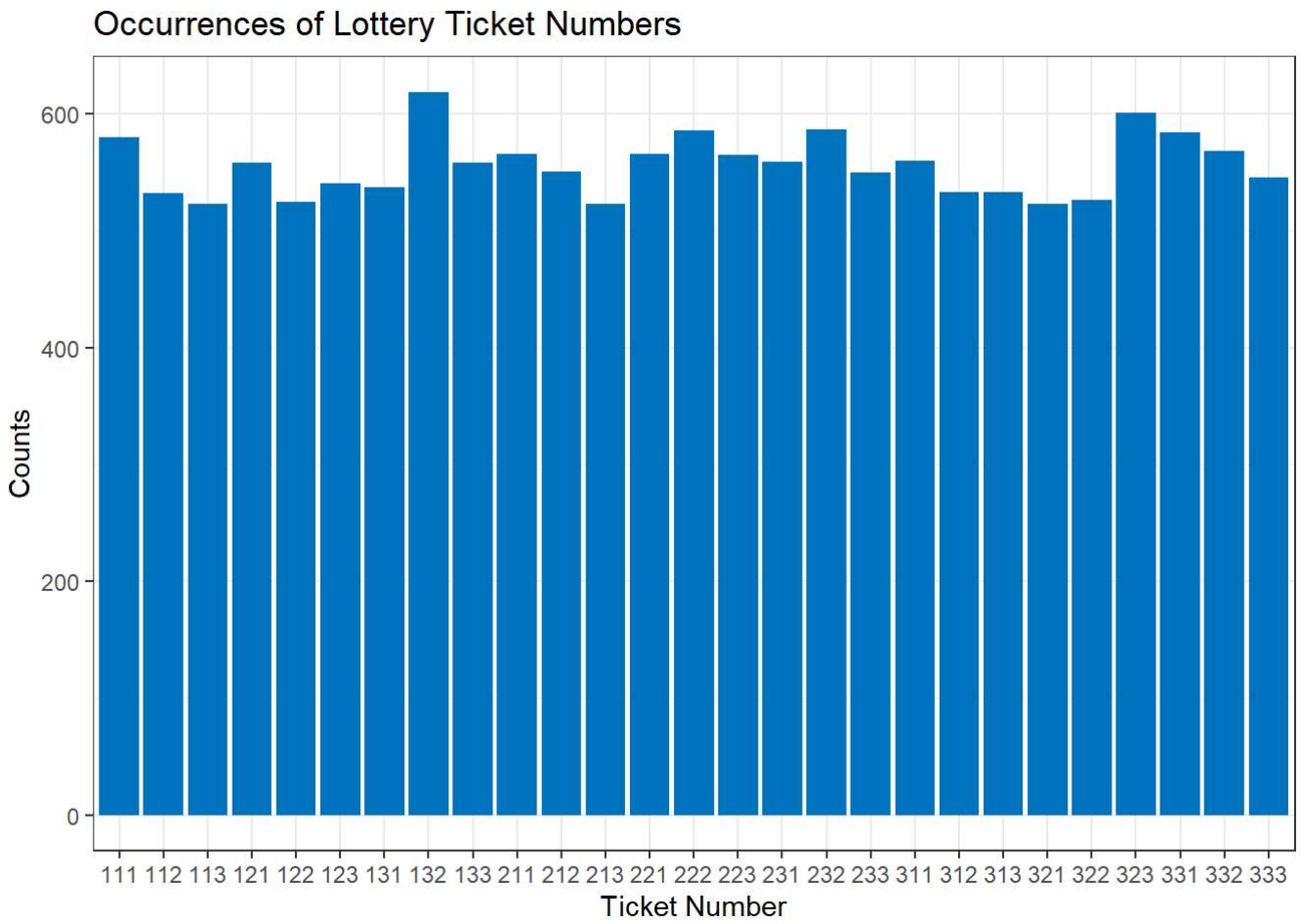
  # Create dataframe
  ticket <- 100*num1 + 10*num2 + num3
  ticket <- as.data.frame(ticket)

  # Count occurrences
  df <- ticket %>%
    group_by(ticket) %>%
    summarise(counts = n())

  # Graph occurrences
  df$ticket <- as.factor(df$ticket)
  p <- ggplot(df, aes(x = ticket, y = counts)) +
       geom_bar(fill = "#0073C2FF", stat = "identity") +
       theme_bw() +
       labs(title = "Occurrences of Lottery Ticket Numbers",
            x = "Ticket Number", y = "Counts")
  print(p)

  return(list(ticket_counts = df, sum_frequencies = table(sum)))
}

lottery()
```




```
## $ticket_counts
## # A tibble: 27 × 2
##    ticket counts
##    <fct>   <int>
##  1 111       580
##  2 112       532
##  3 113       523
##  4 121       558
##  5 122       525
##  6 123       541
##  7 131       537
##  8 132       619
##  9 133       558
## 10 211       566
## # ℹ 17 more rows
##
## $sum_frequencies
## sum
##    3    4    5    6    7    8    9
##  580 1656 3262 3884 3353 1719  546
```

The histogram shows the distribution of sums. The expected sum per ticket is 6, the frequency for each sum is determined by the combinations that produce that sum. The simulation indicates that there is a normal distribution when it comes to sums. With the mean being roughly 6.

```
# Set seed
set.seed(67)

n_pi <- 1000000
x_vals <- runif(n_pi, min = -1, max = 1)
g_vals <- sqrt(1 - x_vals^2)

# Estimate E[g(X)] and then estimate pi
pi_over_four <- mean(g_vals)
pi_estimate <- 4 * pi_over_four
pi_estimate
```

```
## [1] 3.14099
```

```
set.seed(67)

n_int <- 1000000
t_vals <- runif(n_int, min = 0, max = 2)
g_t <- sqrt(1 + t_vals^2 + t_vals^4)

int_est <- 2 * mean(g_t)
int_est
```

```
## [1] 4.193471
```

```
set.seed(67)

n_binom <- 10000
X_sim <- rbinom(n_binom, size = 21, prob = 3/5)
Y_sim <- rbinom(n_binom, size = 19, prob = 3/5)
sum_sim <- X_sim + Y_sim

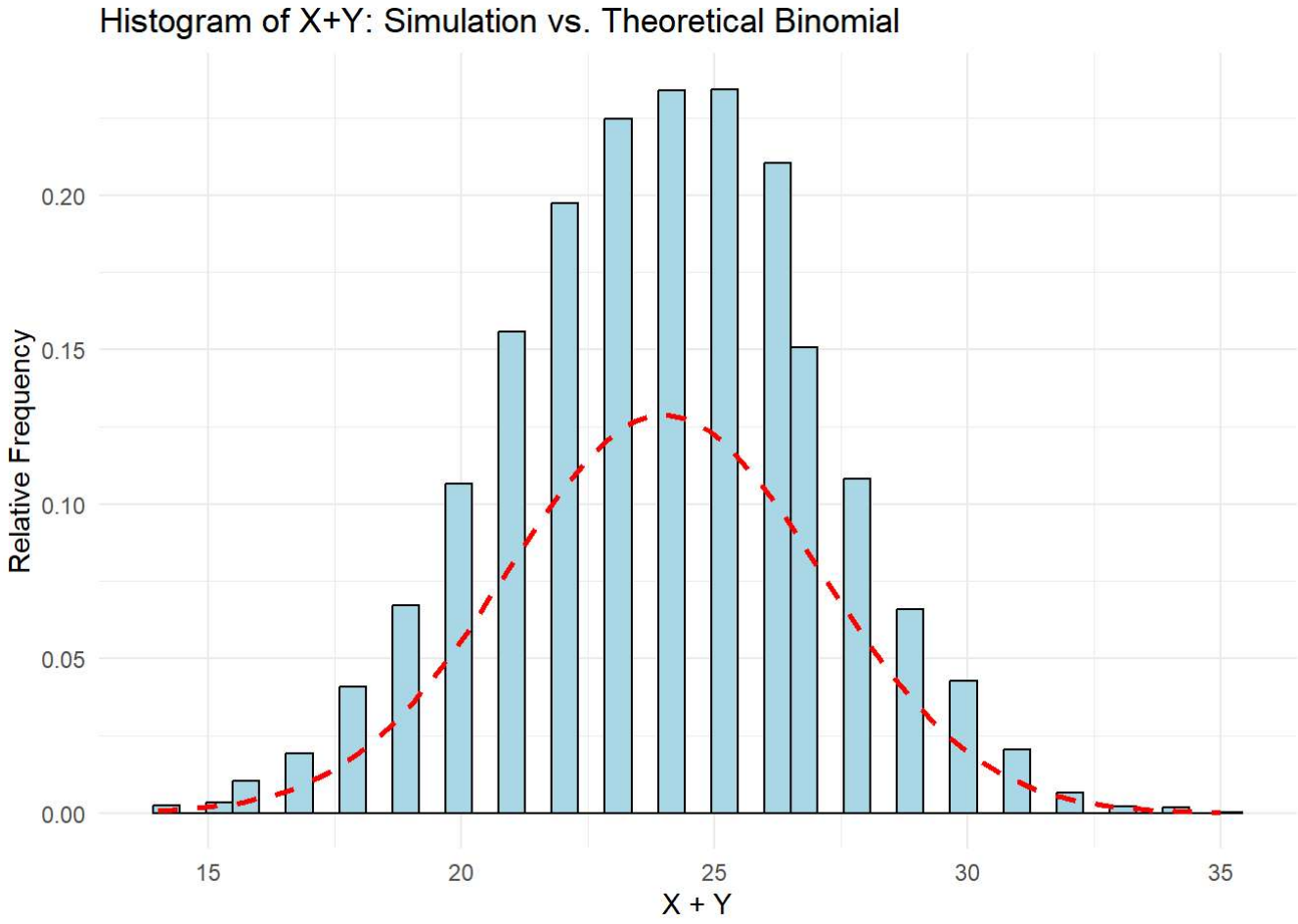
# Theoretical distribution
theo_probs <- dbinom(0:40, size = 40, prob = 3/5)

# Create a data frame
binom_df <- data.frame(Sum = sum_sim)

# Plot histogram
ggplot(binom_df, aes(x = Sum)) +
  geom_histogram(aes(y = ..density..), bins = 41, fill = "lightblue", color = "black") +
  stat_function(fun = function(x) { dnorm(x, mean = 40 * 3/5, sd = sqrt(40 * 3/5 * 2/5)) },
                color = "red", linetype = "dashed", size = 1) +
  labs(title = "Histogram of X+Y: Simulation vs. Theoretical Binomial",
       x = "X + Y",
       y = "Relative Frequency") +
  theme_minimal()
```

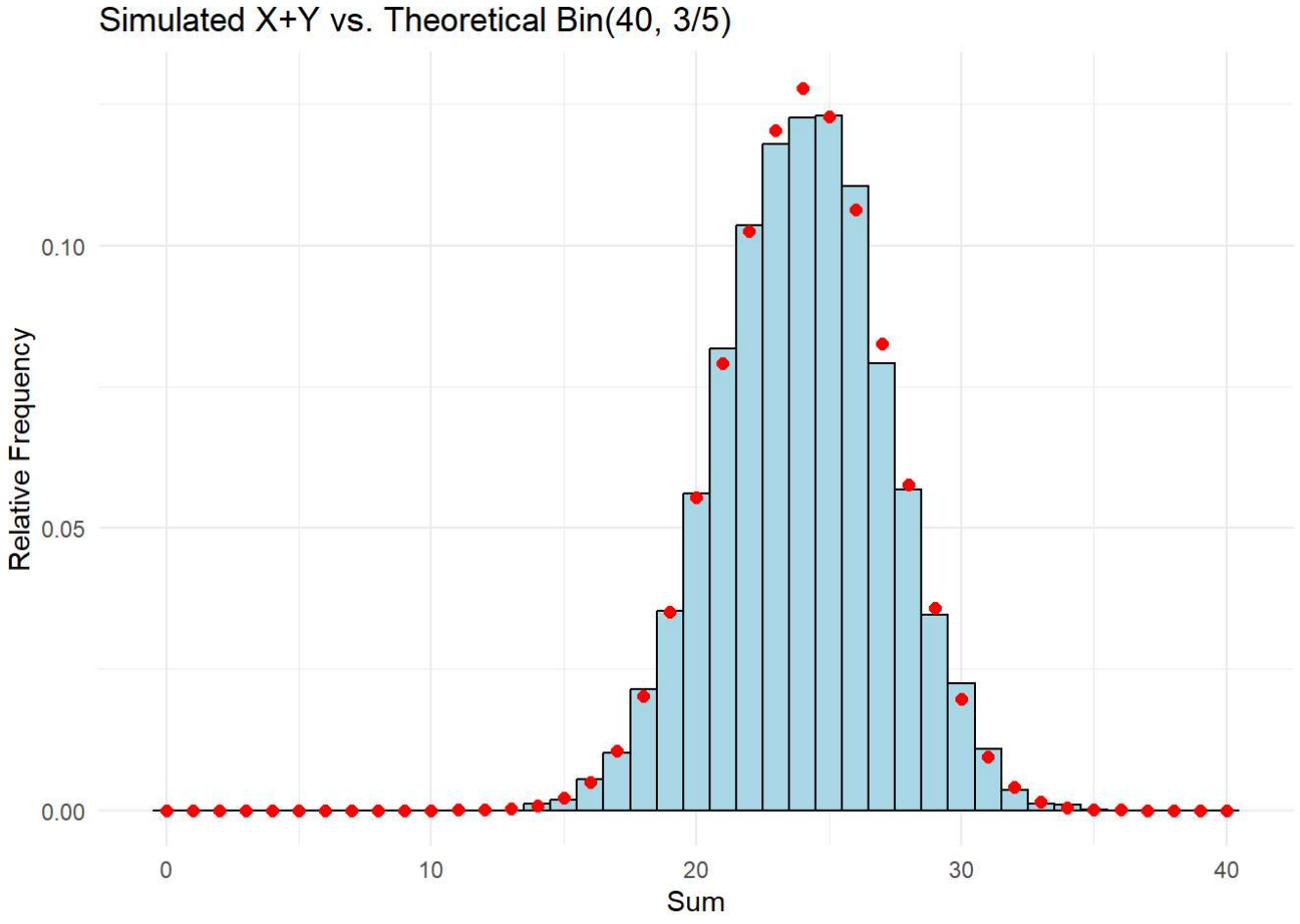
```
## Warning: Using `size` aesthetic for lines was deprecated in ggplot2 3.4.0.
## ℹ Please use `linewidth` instead.
## This warning is displayed once every 8 hours.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```

```
## Warning: The dot-dot notation (`..density..`) was deprecated in ggplot2 3.4.0.
## ℹ Please use `after_stat(density)` instead.
## This warning is displayed once every 8 hours.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```



```
theo_df <- data.frame(
  Sum = 0:40,
  Theoretical = theo_probs
)

ggplot() +
  geom_bar(data = binom_df, aes(x = Sum, y = ..density..),
           stat="bin", bins = 41, fill = "lightblue", color = "black") +
  geom_point(data = theo_df, aes(x = Sum, y = Theoretical), color = "red", size = 2) +
  labs(title = "Simulated X+Y vs. Theoretical Bin(40, 3/5)",
       x = "Sum",
       y = "Relative Frequency") +
  theme_minimal()
```



The histogram for X + Y from the simulation closely resembles the theoretical binomial. This shows that if the simulation was run an infinite amount of times then the sum of the 2 independent binomials is going to be Bin(40,0.6)